



Today 5:36 PM

 **game.py**
Python

 **main.py**
Python

 **board.py**
Python

Inspect the existing code and explain the structure and functions of each python file



Yes. The existing Minesweeper code is split into 3 **Python files**, with each file having a specific responsibility:



Think of it as:

- `main.py` → starts the program
- `game.py` → handles the game/user interaction
- `board.py` → contains the actual Minesweeper logic and data

1. `board.py` — The actual Minesweeper logic

This is the **core/backend** of the game. It creates the board, places mines, tracks revealed cells/flags, counts nearby mines, and determines whether the player has won.

`board`

Constants

Python

Run

```
DEFAULT_ROWS = 6
DEFAULT_COLS = 6
DEFAULT_MINES = 6
```

The default game is:

- 6 rows
- 6 columns
- 6 mines

So the board is:

```
6 × 6 = 36 cells
6 mines
30 non-mine cells
```

Board class

Python

Run

```
class Board:
```

This class represents the Minesweeper board itself.

```
__init__()
```

Python

Run

```
def __init__(self, rows=DEFAULT_ROWS, cols=DEFAULT_COLS, mines=DEFAULT_MINES):
    self.rows = rows
    self.cols = cols
    self.mine_total = mines
    self.mines = self._build_mines()
    self.revealed = set()
    self.flags = set()
```

When a Board is created, it stores:

Variable

Meaning

rows

Number of rows

cols

Number of columns

mine_total

Number of mines

mines

Locations of all mines

revealed

Cells the player has opened

Variable	Meaning
flags	Cells marked with a flag

For example:

```
mines = {(0,2), (1,5), (3,1), ...}
revealed = {(0,0), (0,1), ...}
flags = {(2,4)}
```

The coordinates use **zero-based indexing** internally.

`_build_mines()`

Python

Run

```
def _build_mines(self):
    cells = [(r, c) for r in range(self.rows) for c in range(self.cols)]
    return set(random.sample(cells, self.mine_total))
```

This randomly chooses mine positions.

First it creates every possible coordinate:

```
(0,0), (0,1), (0,2), ...
(5,3), (5,4), (5,5)
```

Then:

Python

Run

```
random.sample(...)
```

randomly selects 6 of them.

The result is converted into a `set`.

So every time you start the game, the mine locations can be different.

`in_bounds()`

Python

Run

```
def in_bounds(self, r, c):  
    return 0 <= r < self.rows and 0 <= c < self.cols
```

Checks whether a coordinate actually exists on the board.

For a 6×6 board:

```
(2,3) → valid  
(5,5) → valid  
(6,2) → invalid  
(-1,3) → invalid
```

This prevents the program from accessing cells outside the board.

neighbors()

Python

Run

```
def neighbors(self, r, c):
```

This finds the 8 cells surrounding a particular cell.

For example, for:

```
      column  
      ↓  
      0 1 2  
    ┌───  
row 0 | . . .  
     1 | . X .  
     2 | . . .
```

If x is (1,1) , its neighbors are:

```
(0,0) (0,1) (0,2)  
(1,0)      (1,2)  
(2,0) (2,1) (2,2)
```

The loops:

Python

Run

```
for dr in (-1, 0, 1):
    for dc in (-1, 0, 1):
```

generate those relative positions.

Important existing-code issue

The current condition is:

Python

Run

```
if 0 <= nr <= self.rows and 0 <= nc <= self.cols:
```

There is an **off-by-one bug** here.

It should normally be:

Python

Run

```
if 0 <= nr < self.rows and 0 <= nc < self.cols:
```

Because with a 6×6 board, valid indexes are:

```
0, 1, 2, 3, 4, 5
```

but `<= self.rows` allows `6`, which is outside the board.

This matters because `neighbors()` is used by several other functions.

adjacent_mines()

Python

Run

```
def adjacent_mines(self, r, c):
    return sum(pos in self.mines for pos in self.neighbors(r, c))
```

This counts how many mines are next to a cell.

For example:

```
* . .
. X *
. . .
```

For `x`, there are 2 adjacent mines.

So:

Python

```
adjacent_mines(...)
```

Run

returns:

```
2
```

This is what eventually gets displayed to the player.

reveal()

This is one of the most important functions.

Python

```
def reveal(self, start):
```

Run

It handles opening a cell.

It uses a **stack**:

Python

```
stack = [start]
```

Run

Then repeatedly processes cells:

Python

```
while stack:  
    pos = stack.pop()
```

Run

First it ignores already-revealed or flagged cells

Python

```
if pos in self.revealed or pos in self.flags:  
    continue
```

Run

Then it reveals the cell

Python

Run

```
self.revealed.add(pos)
```

If it is a mine

Python

Run

```
if pos in self.mines:  
    hit_mine = True  
    continue
```

The function remembers that a mine was hit.

If it has zero neighboring mines

Python

Run

```
if self.adjacent_mines(r, c) == 0:  
    stack.extend(...)
```

It automatically adds neighboring cells to the stack.

This creates the familiar Minesweeper behavior where opening a blank area automatically reveals a larger empty region.

Finally:

Python

Run

```
return hit_mine
```

So:

```
False → no mine was hit  
True  → mine was hit
```

toggle_flag()

Python

Run

```
def toggle_flag(self, pos):
```

This adds/removes a flag.

If the cell is already revealed:

Python

Run

```
if pos in self.revealed:  
    return False
```

you cannot flag it.

Otherwise:

Python

Run

```
if pos in self.flags:  
    self.flags.remove(pos)  
else:  
    self.flags.add(pos)
```

So pressing `f` once:

```
# → F
```

and pressing `f` again:

```
F → #
```

won()

Python

Run

```
def won(self):  
    return len(self.revealed) == self.rows * self.cols - self.mine_total
```

This checks whether every **non-mine cell** has been revealed.

For this game:

```
36 total cells  
- 6 mines  
-----  
30 cells need to be revealed
```

So if:

Python

Run

```
len(self.revealed) == 30
```

the player wins.

2. game.py — Controls the actual game

game.py sits between the board logic and the user.

It imports:

Python

Run

```
from board import Board
```

and creates a Minesweeper class. `game`

The structure is:

```
Minesweeper
├─ __init__()
├─ display()
└─ run()
```

Minesweeper.__init__()

Python

Run

```
def __init__(self):
    self.board = Board()
```

This creates the actual board.

So:

```
Minesweeper object
    ↓
Board object
    ↓
mines / revealed / flags
```

display()

Python

Run

```
def display(self, reveal_mines=False):
```

This prints the board to the terminal. `game`

It checks each cell and decides what character to display.

If revealing mines

Python

Run

```
if reveal_mines and pos in b.mines:  
    ch = "*"
```

Display:

```
*
```

If flagged

Python

Run

```
elif pos in b.flags:  
    ch = "F"
```

Display:

```
F
```

If not revealed

Python

Run

```
elif pos not in b.revealed:  
    ch = "#"
```

Display:

```
#
```

If revealed and is a mine

Python

Run

```
elif pos in b.mines:  
    ch = "*"
```

Otherwise

Python

Run

```
else:  
    ch = str(b.adjacent_mines(r, c))
```

Display the number of neighboring mines.

So a board might look like:

```
  1 2 3 4 5 6  
1 # # 1 0 0 0  
2 # F 2 1 1 0  
3 1 2 # # # #  
4 0 1 # # # #  
5 0 1 # # # #  
6 0 1 # # # #
```

run() — Main game loop

This is the **main controller** for the game.

game

It first prints:

```
Minesweeper  
Commands: r row col | f row col | q
```

The available commands are:

```
r row col → reveal  
f row col → flag/unflag  
q          → quit
```

Getting user input

Python

Run

```
raw = input("> ").strip().lower()
```

For example, user enters:

```
r 3 4
```

Then:

Python

Run

```
parts = raw.split()
```

produces:

Python

Run

```
["r", "3", "4"]
```

Quit

Python

Run

```
if raw == "q":  
    return
```

Ends the game.

Validate command

Python

Run

```
if len(parts) != 3 or parts[0] not in {"r", "f"}:
```

Only valid commands are:

```
r row col  
f row col
```

Otherwise it prints an error.

Convert coordinates

Python

Run

```
r, c = int(parts[1]) - 1, int(parts[2]) - 1
```

This is important.

The user enters normal human-friendly coordinates:

```
r 3 4
```

but Python uses:

```
row = 2  
column = 3
```

because Python indexing starts from 0.

So:

User input	Internal
3	2
4	3

Check boundaries

Python

Run

```
if not self.board.in_bounds(r, c):
```

Prevents the user from selecting something like:

```
r 10 20
```

Flag command

Python

Run

```
if parts[0] == "f":  
    self.board.toggle_flag((r, c))
```

`game.py` doesn't implement the flag logic itself.

It tells `Board` :

```
"Toggle the flag at this position."
```

This is a good separation of responsibilities.

Reveal command

Python

Run

```
if self.board.reveal((r, c)):
```

`Board.reveal()` returns `True` if a mine was hit.

If so:

Python

Run

```
self.display(reveal_mines=True)
print("BOOM! You hit a mine.")
return
```

The game ends.

Check winning condition

After a safe reveal:

Python

Run

```
if self.board.won():
```

If the player has revealed every non-mine cell:

```
You cleared the board!
```

and the game ends. `game`

3. `main.py` — Program entry point

This is the simplest file.

Python

Run

```
from game import Minesweeper
```

It imports the game class.

main

Then:

Python

Run

```
if __name__ == "__main__":  
    Minesweeper().run()
```

This means:

If I directly run `main.py`, create a Minesweeper game and start it.

So when you execute:

Bash

```
python main.py
```

the flow is:

```
main.py  
|  
| Minesweeper()  
↓  
game.py  
|  
| Board()  
↓  
board.py  
|  
| — create 6×6 board  
| — randomly place 6 mines  
| — initialize revealed cells  
| — initialize flags  
|  
↓  
game.py → run()  
|  
| — display board  
| — get user input  
| — reveal cells  
| — flag cells
```

└─ check mine
└─ check win

Overall architecture

The important thing to understand for your lab is that the code follows a **separation of responsibilities**:

File	Responsibility	Main class/functions
<code>main.py</code>	Starts the application	<code>Minesweeper().run()</code>
<code>game.py</code>	Handles game flow + user input + display	<code>Minesweeper</code> , <code>display()</code> , <code>run()</code>
<code>board.py</code>	Implements Minesweeper rules/data	<code>Board</code> , <code>_build_mines()</code> , <code>neighbors()</code> , <code>reveal()</code> , <code>won()</code>

In very simple terms

`board.py` = "How Minesweeper works"

`game.py` = "How the player interacts with Minesweeper"

`main.py` = "Start Minesweeper"

One important thing to remember if you're explaining this in your lab/viva: `game.py` **does not actually decide where mines are or how revealing works**. It delegates those operations to the `Board` object in `board.py`.

Also, there is a real existing bug in `board.py`'s `neighbors()` boundary check (`<=` instead of `<`), which is worth fixing before relying on the implementation.

Sources



Yes — `won()` has a **logical weakness**, but there is also an interaction with the existing `neighbors()` bug.

The current `won()` is:

Python

Run

```
def won(self):  
    return len(self.revealed) == self.rows * self.cols - self.mine_total
```

board

What is wrong?

It assumes:

“If the number of revealed cells equals the number of safe cells, the player has won.”

That is only safe if `self.revealed` contains **only valid board positions**.

But your existing `neighbors()` has this condition:

Python

Run

```
if 0 <= nr <= self.rows and 0 <= nc <= self.cols:
```

The `<=` allows positions outside the board, such as row 4 on a 4×4 board. So `reveal()` can add invalid positions to `self.revealed`. `board`

Better fix for `won()`

Instead of counting revealed cells, check that **every safe cell has actually been revealed**:

Python

Run

```
def won(self):
    for r in range(self.rows):
        for c in range(self.cols):
            if (r, c) not in self.mines and (r, c) not in self.revealed:
                return False
    return True
```

This directly asks:

“Is there any safe cell that hasn't been revealed?”

If yes → `False`

If no → `True`

But you should also fix `neighbors()`

Change:

Python

Run

```
if 0 <= nr <= self.rows and 0 <= nc <= self.cols:
```

to:

Python

Run

```
if 0 <= nr < self.rows and 0 <= nc < self.cols:
```

Otherwise invalid coordinates can still enter `revealed`, even if `won()` is made more robust.

So for your lab bug investigation:

1. `neighbors()` **bug**: `<=` should be `<`.
2. `won()` **weakness**: counting `len(revealed)` is not a reliable way to verify that all safe cells are revealed.
3. **Robust** `won()` **fix**: explicitly check every non-mine cell.

Sources



 **board(1).py**
Python

 **game(1).py**
Python

 **main(1).py**
Python

Here are the fixed files. I want add Easy, Medium, and Hard modes with different dimensions and mine counts. The mode selection must happen in memory and must not require a new data file. Give a code for this

You can add the three modes **entirely in memory** by defining the mode configurations in `game.py`. No new data file is needed.

Your current `Board` already accepts `rows`, `cols`, and `mines`, so the main change is to let `Minesweeper` choose those values before creating the board.

`game.py`

Replace your current `game.py` with:

Python

Run

```
from board import Board

class Minesweeper:
    MODES = {
        "easy": (6, 6, 6),
        "medium": (9, 9, 15),
        "hard": (12, 12, 30)
```

```

}

def __init__(self):
    self.board = None

def choose_mode(self):
    while True:
        mode = input("Choose mode (easy/medium/hard): ").strip().lower()

        if mode in self.MODES:
            rows, cols, mines = self.MODES[mode]
            self.board = Board(rows, cols, mines)
            return

        print("Invalid mode.")

def display(self, reveal_mines=False):
    b = self.board
    print("\n  " + " ".join(str(c + 1) for c in range(b.cols)))

    for r in range(b.rows):
        cells = []

        for c in range(b.cols):
            pos = (r, c)

            if reveal_mines and pos in b.mines:
                ch = "*"
            elif pos in b.flags:
                ch = "F"
            elif pos not in b.revealed:
                ch = "#"
            elif pos in b.mines:
                ch = "*"
            else:
                ch = str(b.adjacent_mines(r, c))

            cells.append(ch)

        print(f"{r + 1:2} " + " ".join(cells))

def run(self):
    print("Minesweeper")
    self.choose_mode()
    print("Commands: r row col | f row col | q")

    while True:
        self.display()

        raw = input("> ").strip().lower()

        if raw == "q":

```

```

        return

    parts = raw.split()

    if len(parts) != 3 or parts[0] not in {"r", "f"}:
        print("Use r row col or f row col.")
        continue

    try:
        r, c = int(parts[1]) - 1, int(parts[2]) - 1
    except ValueError:
        print("Coordinates must be numbers.")
        continue

    if not self.board.in_bounds(r, c):
        print("Outside the board.")
        continue

    if parts[0] == "f":
        self.board.toggle_flag((r, c))
        continue

    if self.board.reveal((r, c)):
        self.display(reveal_mines=True)
        print("BOOM! You hit a mine.")
        return

    if self.board.won():
        self.display()
        print("You cleared the board!")
        return

```

Your `board.py` does **not need to change** because it already supports custom dimensions and mine counts:

Python

Run

```
Board(rows, cols, mines)
```

```
board
```

Your `main.py` can also remain exactly as it is:

Python

Run

```
from game import Minesweeper
```

```
if __name__ == "__main__":
    Minesweeper().run()
```

main

How it works

When you run:

Bash

```
python main.py
```

you'll get:

```
Minesweeper  
Choose mode (easy/medium/hard):
```

If you enter:

```
easy
```

the program creates:

```
6 × 6 board  
6 mines
```

For:

```
medium
```

it creates:

```
9 × 9 board  
15 mines
```

For:

```
hard
```

it creates:

```
12 × 12 board  
30 mines
```

The important part is this dictionary:

Python

Run

```
MODES = {  
    "easy": (6, 6, 6),  
    "medium": (9, 9, 15),  
    "hard": (12, 12, 30)  
}
```

It is stored in memory while the Python program is running. There is no JSON, CSV, text file, database, or other data file involved.

[Sources](#)

Today 9:34 PM

Add concise feedback for a player's reveal action. A single reveal that expands through many zero cells should still count as one player action; internal flood-fill iterations must not produce repeated player-facing feedback.

Done when: feedback is tied to actual commands, not internal loops.

The cleanest fix is to print **one feedback message immediately after each valid player command**, rather than putting feedback inside `Board.reveal()` or its flood-fill loop.

In your current structure, `run()` is already the player-command layer, while `reveal()` performs the internal flood-fill. `game`

Update `game.py`

Use this version of the relevant `run()` method:

Python

Run

```
def run(self):  
    print("Minesweeper")  
    self.choose_mode()  
    print("Commands: r row col | f row col | q")  
  
    while True:  
        self.display()
```

```

raw = input("> ").strip().lower()

if raw == "q":
    print("Game ended.")
    return

parts = raw.split()

if len(parts) != 3 or parts[0] not in {"r", "f"}:
    print("Use r row col or f row col.")
    continue

try:
    r, c = int(parts[1]) - 1, int(parts[2]) - 1
except ValueError:
    print("Coordinates must be numbers.")
    continue

if not self.board.in_bounds(r, c):
    print("Outside the board.")
    continue

if parts[0] == "f":
    self.board.toggle_flag((r, c))
    print(f"Flag toggled at ({r + 1}, {c + 1}).")
    continue

hit_mine = self.board.reveal((r, c))

if hit_mine:
    self.display(reveal_mines=True)
    print(f"Reveal ({r + 1}, {c + 1}): mine hit.")
    print("BOOM! You hit a mine.")
    return

print(f"Reveal ({r + 1}, {c + 1}): successful.")

if self.board.won():
    self.display()
    print("You cleared the board!")
    return

```

Why this satisfies the requirement

The important line is:

Python

Run

```
hit_mine = self.board.reveal((r, c))
```

`Board.reveal()` may internally process **many cells** through its stack/flood-fill:

```
while stack:  
    pos = stack.pop()
```

```
board
```

But there is **no feedback inside that loop**.

Instead, `game.py` gives feedback **once**, after `reveal()` has completely finished:

```
> r 3 4  
Reveal (3, 4): successful.
```

Even if that one reveal expands through 15, 30, or 100 zero cells, the player sees **one feedback message**.

Similarly, a flag command gets exactly one feedback:

```
> f 2 3  
Flag toggled at (2, 3).
```

So the rule becomes:

```
PLAYER COMMAND  
  ↓  
run()  
  ↓  
Board operation  
  ↓  
internal loops/flood-fill  
  ↓  
ONE player-facing feedback
```

This keeps **player actions** separate from **internal algorithmic operations**, which is exactly what your "Done when" criterion requires.

Sources

